

Capstone Project Requirement Document

Phase 1 — Helpdesk Ticket Management System

1. Project Title

Helpdesk Ticket Management System

2. Project Overview

Modern organizations rely heavily on internal IT support teams to resolve employee technical issues such as VPN failures, software installation requests, laptop issues, email access problems, and password resets.

In many organizations, these requests are still handled manually through emails, spreadsheets, or chat messages, resulting in:

- Poor ticket tracking
- Delayed resolutions
- Duplicate issue handling
- Lack of historical visibility
- No centralized knowledge repository

To address these challenges, the organization wants to build a centralized web-based **Helpdesk Ticket Management System** using:

- **Frontend:** React
- **Backend:** Python FastAPI
- **Database:** SQLite/PostgreSQL

This Phase 1 implementation focuses on building a foundational full-stack application with core ticket management capabilities. The architecture should also support future enhancements involving:

- Data Engineering pipelines
- Analytics dashboards
- AI-powered semantic search
- RAG-based enterprise support assistants

3. Business Problem Statement

The organization currently lacks a centralized mechanism for managing internal employee support requests.

Current challenges include:

- Employees repeatedly reporting similar issues
- No structured tracking of tickets

- Difficulty monitoring ticket status
- No searchable repository of previous resolutions
- Inefficient support workflows
- Lack of analytics on recurring issues

The proposed application will streamline the support lifecycle by digitizing ticket creation, management, and resolution tracking.

4. Project Objective

The objective of this capstone project is to design and develop a basic but scalable Helpdesk Ticket Management System that enables employees and support teams to:

- Create support tickets
- Track ticket status
- Manage issue resolution workflows
- Search existing tickets
- Maintain historical support records

The system should follow clean architecture principles and REST API design standards.

5. Scope of Phase 1

Phase 1 includes:

- Full-stack application development
- REST API implementation
- Ticket CRUD operations
- Ticket search functionality
- Database integration
- Frontend-backend integration

Phase 1 does NOT include:

- Authentication/authorization
- Notifications
- AI/ML integration
- Analytics dashboards
- Semantic search
- Cloud deployment

6. Functional Requirements

6.1 Ticket Creation

The system should allow employees to create support tickets by providing:

- Employee Name
- Department
- Issue Category
- Ticket Description
- Priority Level

6.2 View Tickets

The system should allow users to:

- View all tickets
- View ticket details
- View ticket status
- View ticket creation timestamp

6.3 Update Ticket

Support administrators should be able to:

- Update ticket status
- Modify issue details
- Add resolution notes

6.4 Delete Ticket

The system should allow deletion of tickets when necessary.

6.5 Search Tickets

The application should support:

- Keyword-based search
- Category-based filtering
- Status-based filtering

7. Ticket Categories

Sample categories include:

- VPN Issue
- Password Reset
- Software Installation
- Laptop Issue
- Email Access

- Network Connectivity
- Hardware Request

8. Ticket Priority Levels

The system should support:

- Low
- Medium
- High
- Critical

9. Ticket Status Values

Supported statuses:

- Open
- In Progress
- Resolved
- Closed

10. System Users

User Type	Responsibilities
Employee	Create and view tickets
Support Admin	Manage and resolve tickets

11. Technical Requirements

11.1 Frontend Requirements (React)

The frontend application should be developed using React.

Mandatory Concepts

- Functional Components
- React Hooks
- Component-based architecture
- Axios or Fetch API
- Form validation
- State management using useState/useEffect

Suggested Pages

Dashboard Page

Displays:

- Total tickets
- Ticket summary
- Recent tickets

Create Ticket Page

Allows employees to raise new support requests.

Ticket Listing Page

Displays all tickets in tabular/card format.

Ticket Details Page

Displays complete information about a ticket.

Search & Filter Section

Allows filtering based on:

- Status
- Category
- Priority

11.2 Backend Requirements (FastAPI)

The backend application should be developed using FastAPI.

Mandatory Concepts

- FastAPI routing
- Pydantic models
- CRUD operations
- Database integration
- Exception handling
- REST API standards
- CORS middleware

12. API Requirements

Method	Endpoint	Description
GET	/tickets	Retrieve all tickets
GET	/tickets/{id}	Retrieve ticket by ID
POST	/tickets	Create ticket
PUT	/tickets/{id}	Update ticket

Method	Endpoint	Description
DELETE	/tickets/{id}	Delete ticket
GET	/search	Search tickets

13. Database Requirements

The application should use:

- SQLite (preferred for Phase 1)
OR
- PostgreSQL

14. Database Schema

Tickets Table

Column Name	Data Type
ticket_id	Integer
employee_name	String
department	String
issue_category	String
description	Text
priority	String
status	String
resolution_notes	Text
created_at	DateTime

15. Suggested Project Structure

Backend Structure

```

backend/
|
├── main.py
├── database.py
├── models.py
├── schemas.py
├── crud.py
├── routers/
|   └── tickets.py
├── services/
└── requirements.txt

```

Frontend Structure

```
frontend/  
|  
├── src/  
|   ├── components/  
|   ├── pages/  
|   ├── services/  
|   ├── App.js  
|   └── api.js
```

16. Non-Functional Requirements

The application should:

- Be responsive
- Handle invalid inputs gracefully
- Follow clean UI design principles
- Maintain modular code structure
- Support future scalability

17. Expected Deliverables

Participants are expected to submit:

Source Code

- Complete React frontend
- Complete FastAPI backend

Database

- Database schema/scripts

Documentation

- README file
- Setup instructions
- API documentation

Screenshots

- Application screenshots
- API testing screenshots

18. Evaluation Criteria

Criteria	Weightage
Frontend Development	20%

Criteria	Weightage
Backend API Development	25%
Database Integration	15%
CRUD Functionality	15%
UI/UX Design	10%
Code Structure & Standards	10%
Documentation	5%

19. Recommended Technology Stack

Layer	Technology
Frontend	React
Backend	FastAPI
Database	SQLite/PostgreSQL
API Testing	Postman
Version Control	Git/GitHub